

STEP Protocol-Based File Upload Service: Design, Implementation, and Performance Evaluation

Binyu Li

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Binyu.Li23@student.xjtlu.edu.cn
Student ID: 2364567

Junyan He

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Junyan.He23@student.xjtlu.edu.cn
Student ID: 2363304

Chenyu Du

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Chenyu.Du23@student.xjtlu.edu.cn
Student ID: 2360743

Zengyi Mei

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Zengyi.Mei23@student.xjtlu.edu.cn
Student ID: 2361796

Jiafei Sun

*School of Advanced Technology
Xi'an Jiaotong Liverpool University
Suzhou, China*

Jiafei.Sun23@student.xjtlu.edu.cn
Student ID: 2361510



CAN201 - GROUP 29

Advisors: Dr. Wenjun Fan, Dr. Fei Cheng, Dr. Gordon Boateng

Abstract—This paper presents the design and evaluation of a Python-based STEP client for the CAN201 networking coursework. The system supports token-based authentication, multi-threaded block-wise uploads, and end-to-end integrity verification over TCP. We focus on application-layer protocol design, addressing challenges in concurrency, error handling, and reliability. The resulting system demonstrates practical techniques used in modern networked file transfer and cloud storage services.

Index Terms—networking, file transfer, TCP, authentication, STEP protocol, multi-threading

I. INTRODUCTION

Fast and reliable file exchange is crucial for cloud storage, collaboration platforms, and large-scale content delivery. In the CAN201 coursework, this is explored using the STEP protocol, which transmits JSON control messages and binary file blocks over TCP. Students are tasked with completing an incomplete reference implementation to create a functional client-server system supporting authentication, upload planning, block-wise transfer, and integrity verification.

The primary challenge is repairing the server to properly parse STEP packets, handle connections, and enforce token-based authorization. On the client side, the focus is on implementing a socket-based uploader that handles login, negotiates upload plans via `SAVE`, splits files into blocks, and ensures reliable transmission, including re-authentication, timeout handling, and MD5 correctness checks. A multi-threaded upload strategy and support for breakpoint-resume via `STATUS` are also implemented for efficiency.

These mechanisms reflect real-world practices in cloud storage and large-file distribution, such as token-controlled access, resumable uploads, and end-to-end hashing. Implementing them over TCP reveals challenges like framing, partial reads, concurrency, and failure recovery.

In this project, we develop a STEP-compliant client and an improved server that together provide a robust, resumable file upload service. Our work includes fixing server-side protocol handling, implementing multi-threaded block uploads, integrating file-level MD5 verification for uploads and block-level MD5 verification in the extended download function, enabling automatic re-authentication and resume, and evaluating performance across different file sizes and configurations.

II. RELATED WORK

File transfer over the Internet has traditionally relied on application-layer protocols like FTP, HTTP, SFTP, and HTTPS, typically built on TCP to leverage its reliable byte stream. These protocols add command semantics, authentication, and data framing strategies. For instance, HTTP-based upload services use chunked transfer encoding, resumable uploads, and checksum verification, similar to the STEP protocol used in this coursework, where JSON control messages and binary file blocks are multiplexed over a TCP connection [1], [2].

As Internet bandwidth-delay products increase, various transport-layer enhancements have been introduced to better utilize capacity in heterogeneous networks. High-speed congestion control algorithms like CUBIC and BBR stabilize throughput while ensuring fairness and responsiveness under varying congestion [3], [4]. These improvements are often combined with application-layer chunking, multi-threaded clients, adaptive

retries, and resumable uploads to improve performance for large-file transfer and content distribution.

In this project, the STEP-based system can be viewed as a pedagogical microcosm of these techniques: running over standard TCP, it introduces token-based authentication, upload planning, and integrity verification. Our implementation also explores practical enhancements such as multi-threaded uploads, block-level MD5 verification, breakpoint-resume via `STATUS`, and optional pre-upload compression, similar to techniques used in modern cloud storage systems.

III. DESIGN

A. C/S Architecture Overview

This project implements a file upload system based on the Client-Server (C/S) architecture, enabling centralized control, efficient management, and high security. Communication between the client and server follows the Simple Transfer and Exchange Protocol (STEP), a TCP-based request/response protocol for reliable file and data exchange. The diagram in Fig. 9 is in the Appendix.

The server manages connections, dispatches requests, handles token-based authentication, and processes file I/O. It binds to TCP port 1379 and supports multiple simultaneous connections through thread-based concurrency. Each request is validated by message fields. The server generates a token — a temporary digital pass issued after successful login — and checks it in every request.

The client is a multithreaded program that connects to the server, performs authentication with an MD5 hash of the user ID, obtains a token, and requests an upload plan. The file is divided into multiple blocks that include MD5 checksum and uploaded concurrently using parallel threads, with the bandwidth estimation and progress bars displayed in real time.

B. Workflow of Program

The workflow follows the STEP protocol, ensuring reliable and secure data transfer between clients and the server. The process consists of five major phases, and the detailed workflow is illustrated in Fig. 10.

a) Authorization and Token

- 1) When the client starts, it establishes a TCP connection on port 1379 and sends a `LOGIN` request.
 - The request type is `AUTH`, containing the username (student ID) and its MD5 hash as the password.
- 2) After verifying the credentials, the server generates a token to confirm the client's identity.
 - The token includes the username, creation time, expiration time, and a signature for security. It has a limited lifetime (TTL) and is validated in every request to prevent misuse.
- 3) The client stores the token and attaches it to subsequent STEP messages, removing the need for repeated logins.

b) File Uploading

- 1) After authentication, the client starts uploading by sending a `SAVE` request with the filename (key) and file size (size).
- 2) The server responds with an upload plan that defines the file identifier, block size, and total number of blocks.

- 3) The client uploads block in parallel using up to sixteen threads through the `UPLOAD` operation.
- 4) After all blocks are received, the server reconstructs the file, computes its MD5.
 - The client shows progress and bandwidth via `tqdm` and records performance data.

c) File Verification

After the upload is complete, the server returns the file's final MD5 value. The client compares this hash with its own to verify that the transfer is complete and error-free.

- If the hashes differ, the client reports a data integrity error.

d) Error Handling

Both the client and server handle errors based on the STEP status codes (400–410).

- If a token expires (403), the client automatically re-authenticates, refreshes the token, and retries the failed block.
- Other errors such as missing fields (400), are reported with clear messages to help users resolve issues quickly.

e) Downloading and Delete

The system also supports file downloading and deletion, both following the same STEP protocol and token-based authentication as uploading

1) Download

- The client sends `DOWNLOAD` request with the file key and authentication token.
- The server validates the token and checks whether the file exists.
- If found, the file is divided into blocks, like uploading.

2) Delete

- The client sends a `DELETE` request containing the file key, authentication token, and operation type (`FILE` or `DATA`).
- After validating the token, the server determines the user's workspace and performs deletion with clear status codes.

C. Algorithm

This section presents the key algorithms used in our STEP-based system. Both algorithms are shown in separate client-side and server-side views to match our C/S implementation.

The Login and Token Validation pseudo-code is included here to illustrate how the client authenticates and how the server verifies tokens.

The File Upload algorithm, which covers `SAVE` planning and block transmission, is more detailed and is therefore placed in the Appendix.

IV. IMPLEMENTATION

A. Development Environment

The environment of development is shown in Table I.

Algorithm 1 STEP Login and Token Validation

```

1: Client side:
2: Construct payload with username and MD5 password
3: Send LOGIN request using _send_packet
4: Receive reply (j, _)
5: if reply contains FIELD_TOKEN and status is 200 then
6:   Save TOKEN
7: else
8:   Exit with friendly_error
9: end if
10: Server side:
11: Receive (json_data, _) from client
12: if operation is OP_LOGIN then
13:   if type is not TYPE_AUTH then
14:     Reply "Type error" and continue
15:   end if
16:   if username or password missing then
17:     Reply "Missing field" and continue
18:   end if
19:   if MD5(username) does not match received password then
20:     Reply "Password error" and continue
21:   end if
22:   Generate token
23:   Reply "Login OK" with FIELD_TOKEN
24:   continue
25: end if

```

TABLE I
DEVELOPMENT ENVIRONMENT

Field	Description
Operating System	Microsoft Windows 11
CPU	12th Gen Intel(R) Core(TM) i7-12700H
RAM	16 GB
IDE	JetBrains PyCharm Professional 2022.2.4
Python Version	Python 3.10
Python Library	'hashlib', 'socket', 'json', 'os', 'argparse', 'struct', 'threading', 'time', 'numpy', 'gzip', 'zlib', 'shutil', 'tqdm', 'logging', 'math', 'uuid'

B. Implementation Steps

- **Requirements Analysis and Planning:** In the initial stage, the team analyzed the project specification to identify tasks and system objectives, focusing on understanding the server-client protocol and planning development steps, including server debugging, client implementation, and file transfer testing. This phase set a clear development roadmap and laid the foundation for a modular, maintainable system design.
- **Code Review and Development:** The process began with reviewing the server code provided by the examiner, identifying and fixing several syntax and logic errors to ensure successful execution, confirmed by the message "Server is ready." The client application was then developed based on the protocol, implementing user authentication, token retrieval, and block-based file uploading. A command-line interface was designed to support file upload, download,

and status checking, while unit testing verified the correctness of each function.

- **Optimization and Refinement:** After the main implementation, optimizations were made to improve efficiency and reliability. A resumable transfer mechanism allowed uploads to continue from the last completed block, while token expiration enhanced session security. Multithreading boosted transfer speed, and Gzip/zlib compression reduced bandwidth usage. Progress visualization and performance evaluation were handled with tqdm and NumPy, improving system robustness.
- **Deployment and Testing:** The system was deployed and tested in a controlled virtual environment. Testing included server debugging, user authorization, and file upload verification with MD5 checksum. Various file sizes were used to evaluate data integrity, stability, and transfer speed. The client displayed real-time progress, and results confirmed reliable communication and correct file reconstruction.

To provide a clearer understanding of the implementation process, we have included a flowchart Fig. 1. that outlines the key steps in the workflow.

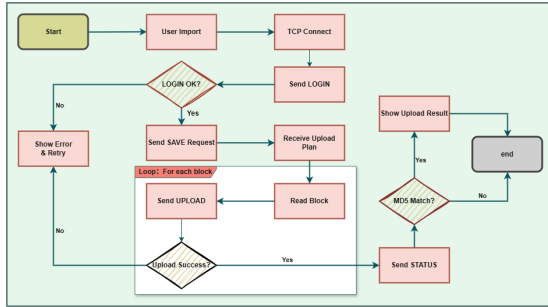


Fig. 1. Program flow of client-server authorization and file uploading

C. Programming Skills

- **Modular Architecture:** The system adopts a modular design that separates STEP protocol handling, file I/O operations, and user interaction logic into distinct functional units. This separation improves code reuse, simplifies maintenance, and makes future extensions easier to implement.
- **Parallel Computing Structure:** The system uses a multi-threaded framework to improve data transmission efficiency. By coordinating concurrent upload and download tasks with Python's threading and semaphore modules, it allows multiple parallel operations. Synchronization locks ensure data consistency and prevent thread conflicts.
- **Authentication and Access Control:** The system employs token-based authentication with a built-in expiration feature to ensure both the security and ongoing validity of user sessions. This mechanism regulates user access by verifying token validity, confining authorized users to fundamental functionalities such as upload and download.
- **Data Transmission Recovery:** The platform allows paused uploads to resume seamlessly from the last completed data unit. logs upload status locally, with file-level

MD5 verification ensuring data integrity. For downloads, each block is validated with block-level MD5 hashes, and after all blocks are received, the system writes the full file to guarantee overall integrity.

- **File Storage (by Key) and Compression Strategy:** The system integrates gzip/zlib compression with file storage and modification based on keys. It saves files via unique keys and supports file updates, while reducing bandwidth consumption and ensuring reliable transfer performance in constrained network environments.
- **Performance Monitoring:** Utilizing tqdm for real-time progress visualization and harnessing NumPy's computational capabilities for performance evaluation, the system provides measurable indicators such as upload speed, bandwidth efficiency, and operational consistency.

V. ACTUAL IMPLEMENTATION

A. Authorization Function

The authorization subsystem is implemented as a token-based client-server protocol on top of the STEP framing format:

- **Connection Establishment:** The client first establishes a TCP connection to the STEP server and uses this socket to send framed JSON requests.
- **Credential Derivation:** For login, the client collects the student ID as the username and derives the password by calculating the MD5 hash of this ID.
- **Protocol Encapsulation for Login:** Authentication is performed via a single LOGIN operation with request type AUTH. The client populates the relevant fields in the JSON body of a STEP request, and the server treats this as a login attempt only after verifying that all required conditions are satisfied.
- **Login Validation and Token Issuance:** The server recomputes MD5(username) and compares it case-insensitively with the supplied password. If successful, it generates a session token containing the username and issuance/expiry timestamps with a configurable TTL; this metadata is signed with a secret MD5 digest, base64-encoded, and returned to the client for expiry verification.
- **Token Propagation, Protection, and Expiry Handling:** After login, the client caches the token and attaches it to all subsequent STEP requests. The server requires a valid token for all operations except LOGIN, returning HTTP 403 for missing or invalid tokens. Token validation checks the signature and expiry; when expired, the client re-authenticates and retries the request, ensuring robustness for long uploads.

B. File Uploading Function

The file uploading function runs after the client has successfully authenticated and obtained a valid session token. The main steps of this function are as follows:

- **Status and Resume:** Before uploading, the client issues a STATUS request with the file key to determine whether the file is absent, complete, or partially uploaded. For partial uploads, the server returns the indices of missing blocks so that only those blocks are retransmitted.

- **Upload Plan Negotiation:** If no upload plan exists, the client sends a SAVE request with the file key and size, and the server returns an upload plan specifying the block size, number of blocks, and associated metadata for subsequent uploads.
- **Block Partitioning and Upload:** Guided by the upload plan, the client divides the file into contiguous blocks, reads each block from disk, and encapsulates it into a STEP frame with operation UPLOAD, type FILE, and a JSON body carrying the file key, block index, and an optional per-block MD5 digest. The raw block bytes are placed in the binary payload of the frame.
- **Multi-threaded Transfer and Robustness:** To accelerate uploads, the client spawns multiple worker threads that upload different blocks in parallel, while socket operations are guarded by a mutex to preserve protocol framing. Threads automatically retry failed transmissions and, upon an HTTP 403 token-expiry response, transparently re-authenticate and resend the affected blocks, ensuring robustness for long-running uploads.
- **Completion and MD5 Verification:** On the server, each valid block is written to a pre-allocated temporary file at the corresponding offset and recorded in a block log; when all planned blocks are received, the server finalizes the file and returns its file-level MD5 hash. The client compares this hash with the pre-computed local MD5 to confirm that the uploaded file is bit-wise identical to the original.

C. Difficulties and Solutions

During implementation, several technical challenges were encountered and addressed systematically.

The first difficulty involved **protocol framing and message parsing**. Early versions of the STEP protocol caused decoding errors due to mixed JSON and binary data. This was solved by adopting a fixed header-payload format using the ‘struct’ library, ensuring reliable communication between client and server.

Another major issue arose in **multi-threaded synchronization**. Concurrent uploads led to race conditions and data inconsistency. Mutex locks and semaphores were introduced to coordinate thread access and guarantee transmission integrity.

The **resumable upload mechanism** also presented challenges in maintaining accurate progress records. A persistent “.log” file that records indices of received blocks was implemented to resume incomplete transfers safely.

Finally, **token expiration** occasionally interrupted ongoing sessions. The client was modified to automatically re-authenticate and continue operations, improving system stability.

Through these refinements, the system achieved stable concurrent transmission, secure authentication, and reliable fault recovery under real network conditions.

VI. TEST AND RESULT

A. Testing environment

The testing environment is shown in Table II.

TABLE II
TESTING ENVIRONMENT OF BOTH SERVER AND CLIENT

Field	Description
Operating System	Ubuntu 20.04.6
CPU	13th Gen Intel(R) Core(TM) i9-13900HX
RAM	16 GB
IDE	JetBrains PyCharm Professional 2024.1.3
Python Version	Python 3.12.4

B. Test Steps

According to the course requirements, after the trial run of the server, we tested the client developed for this server in two virtual environments. The testing process mainly consists of three tasks:

Task 1: Basic Debugging and Testing

- Fixed six syntax errors in server.py
- Verified whether the terminal displays “Server is ready!”

Task 2: Authentication Testing

- Whether the client initiates a request, obtains, and stores the token returned by the server
- Whether the server authenticates the user’s login password and generates a token if the password is correct
- Whether the server can validate the validity of the token in subsequent requests

Task 3: File Transfer and Verification

- Testing the FILE-type SAVE operation to obtain the upload plan, and verify whether the client initiates multiple worker threads via a shared TCP connection for parallel data chunk upload
- Verifying whether both the client and the server support breakpoint resume
- Confirming whether the MD5 checksums of the server and client are consistent
- Verifying that the client collects the upload time of each chunk and estimated bandwidth, and exports the aggregated metrics to a structured log file for subsequent analysis

The following figure shows the results of the basic flow test:

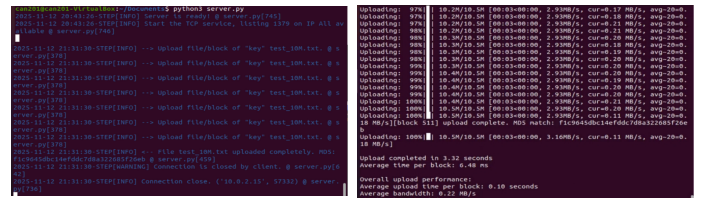


Fig. 2. Server starting and running

Fig. 3. Client running results

C. Testing Results

1) **Comparison of different types of file:** We evaluated upload performance for five file types (txt, jpg, pdf, mp3, mp4) under increasing file sizes, as shown in Fig. 4. For all formats, upload time increases substantially with file size: 10 MB files complete within a few seconds, whereas 250 MB files require approximately 100–120 s. At a fixed file size, differences between file types are modest: text files achieve slightly higher throughput, pdf files are consistently the slowest, and images

(jpg) and media files (mp3, mp4) lie in between. These results indicate that, once file size is controlled, file type has only a secondary impact on upload performance; the primary driver of upload time is the data volume itself.

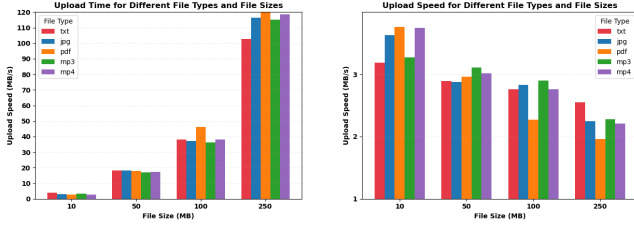


Fig. 4. Comparison of different types of file

2) **Comparison of Different File Sizes:** We examine the impact of file size on upload time and transfer speed, as shown in Fig. 5. Upload time increases approximately linearly as file size grows from 1 MB to 50 MB. Small files (up to 10 MB) upload in a few seconds, while larger files take noticeably longer. Transfer speed remains stable across file sizes, with slight fluctuations and a slight downward trend. This suggests network throughput is near saturation, with increasing upload time driven by data volume rather than bandwidth changes. Consequently, single-threaded uploads provide predictable performance for small and medium files, but the rising latency for larger files motivates exploring optimization strategies such as multithreaded transmission.

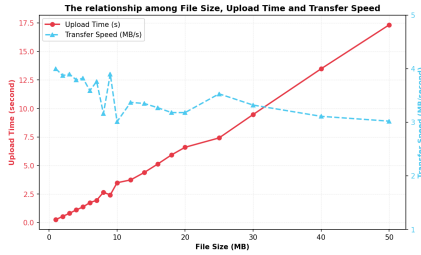


Fig. 5. Relationship of File Size, Upload Time and Transfer Speed

3) **Influence of multi-threaded file upload:** To examine the influence of multithreading on upload performance, we modified the client to transmit files with varying numbers of threads and measured transfer speed and upload time for four file sizes (10 MB, 50 MB, 100 MB, 250 MB), as shown in Fig. 6, 7.

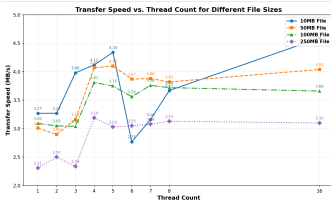


Fig. 6. Transfer Speed vs Threads

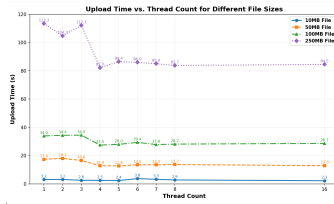


Fig. 7. Upload Time vs Threads

For small files (e.g., 10 MB), increasing the thread count yields only marginal improvements, suggesting that protocol overhead and startup costs dominate. For medium to large files (50–250

MB), however, using a moderate number of threads (around four) noticeably reduces upload time and increases throughput, after which performance saturates or even slightly degrades as additional threads introduce scheduling and synchronization overhead. Overall, these results indicate that multithreading is most beneficial for larger files and that the thread count should be tuned to file size rather than simply maximized.

4) **Impact of Multi-client upload simultaneously:** In practice, file servers are typically accessed by multiple clients concurrently rather than a single uploader. To simulate this, we examined how simultaneous upload requests affect upload speed and transfer time, as shown in Fig. 8. Overall, the system handles a small degree of concurrency with minimal variation in throughput and latency when increasing from one to three clients, but performance degrades with a fourth client, leading to slower upload speeds and longer transfer times. These results indicate that controlling the level of concurrency—e.g., via scheduling or rate-control mechanisms—is important for maintaining stable throughput and predictable upload latency in multi-client scenarios.

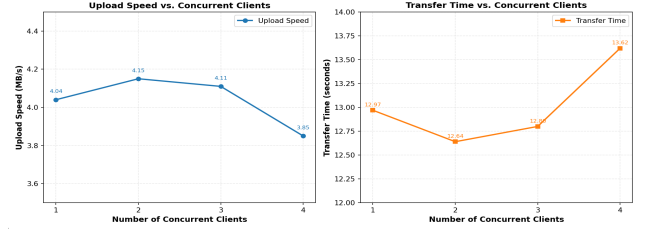


Fig. 8. Multi-client upload simultaneously

VII. CONCLUSION

This project successfully implemented a file transfer system based on the STEP protocol, enabling efficient, secure file uploads with support for resumable transfers. We addressed challenges such as authentication, data integrity verification, and concurrent uploads during development.

The server was debugged to fix protocol parsing and network I/O issues, ensuring proper request processing. The client was implemented with multithreading for improved upload efficiency and file-level MD5 verification to ensure data integrity.

Testing validated the system's performance across various file types and sizes, and optimizations improved efficiency under multithreaded uploads and multiple concurrent clients.

This project strengthened our understanding of protocol design, network programming, debugging, performance testing, and system optimization. It bridged theory with practice, providing a solid foundation for future work in network systems and development.

REFERENCES

- [1] J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," RFC 959, 1985.
- [2] M. Nottingham, "HTTP Semantics," IETF RFC 9110, 2022.
- [3] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008.
- [4] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, and G. Carle, "Towards a deeper understanding of TCP BBR congestion control," in *2018 IFIP Networking Conference (IFIP Networking) and Workshops*, 2018.

APPENDIX

A. C/S Architecture

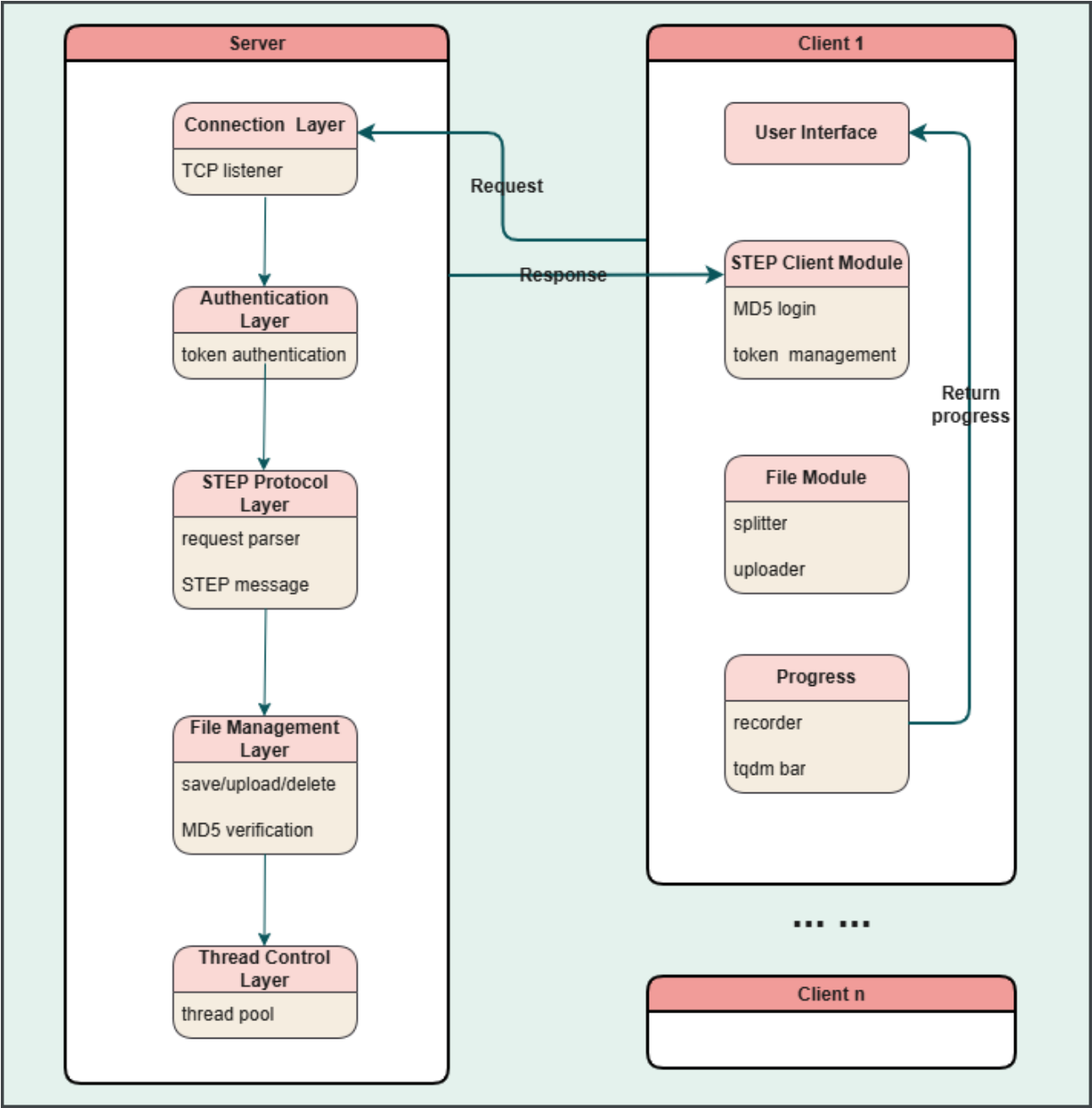


Fig. 9. Client-Server Network Architecture Based on the STEP Protocol

This figure shows the complete C/S architecture presented in the Design section, including the TCP communication layer, STEP message handling, authentication module and file/data manager.

B. Workflow Diagram

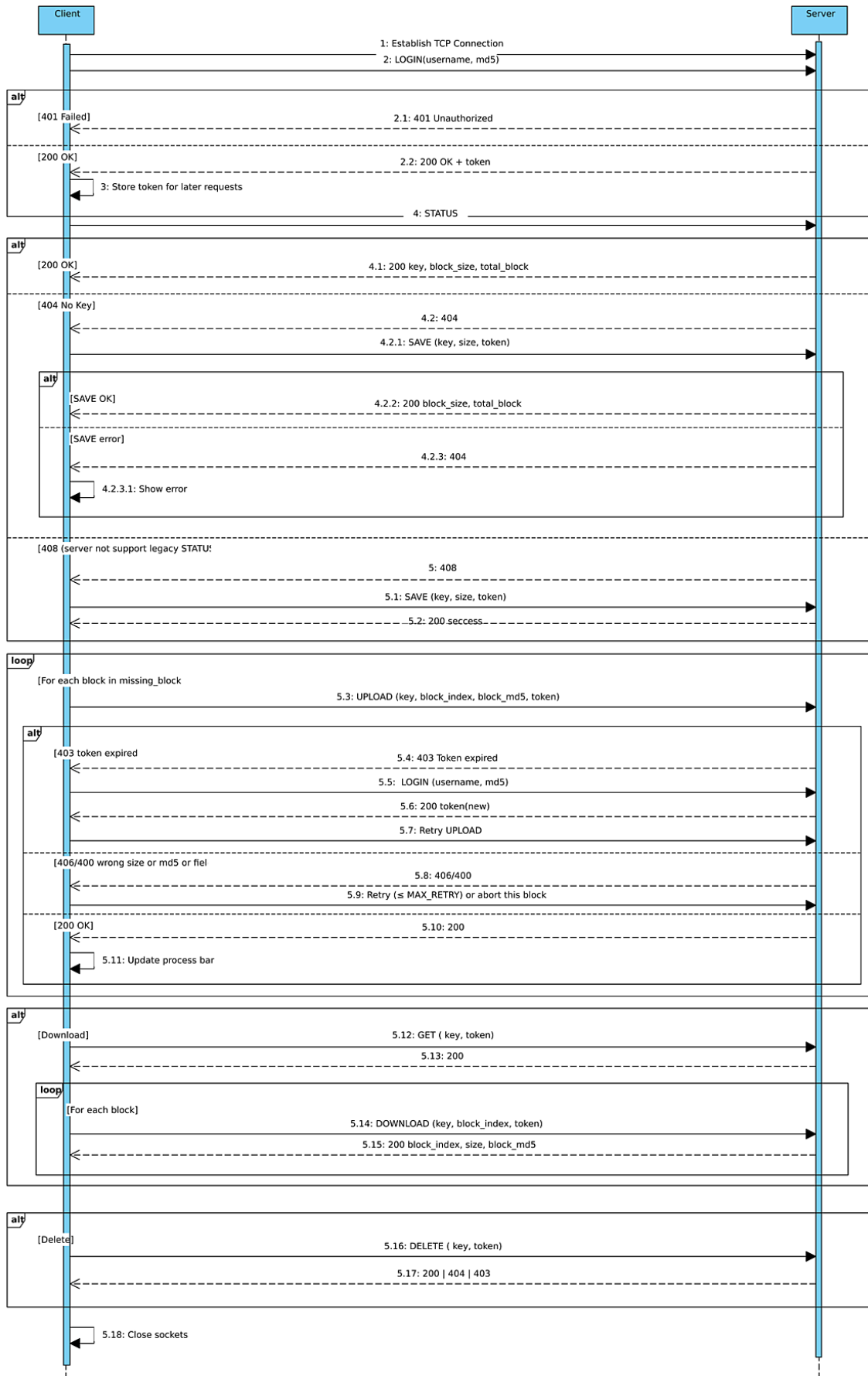


Fig. 10. Complete Workflow of Authorization, Token Issuing, and File Upload

This workflow diagram illustrates the complete STEP client-server interaction, including LOGIN authentication, STATUS checking, SAVE planning, multi-threaded UPLOAD operations, and optional GET/DOWNLOAD/DELETE processes.

C. Algorithm of Upload

Algorithm 2 STEP File Upload (Client & Server)

```
1: Client high-level (simple_upload):
2: Prepare FILE_NAME, FILE_SIZE, LOCAL_MD5
3: Call step_login_operation and create UPLOAD_SOCKET
4: st  $\leftarrow$  step_status_operation(FILE_NAME)
5: if file exists on server then
6:   Determine missing_blocks
7:   if none missing then
8:     return
9:   end if
10: else
11:   Call step_save_operation to create upload plan
12:   missing_blocks  $\leftarrow$  all block indices
13: end if
14: Start _start_upload_threads(missing_blocks)
15: Wait for all threads and finish upload
16: Client per block (step_upload_operation):
17: Read file chunk for block_index
18: for attempt from 1 to MAX_RETRY do
19:   Build payload with FIELD_KEY and FIELD_BLOCK_INDEX
20:   Send OP_UPLOAD request and receive reply
21:   if upload accepted then
22:     return
23:   end if
24:   if token expired then
25:     reauth and retry
26:   end if
27: end for
28: Raise error for failed block
29: Server OP_SAVE (file_process):
30: if request_operation = OP_SAVE then
31:   Validate key and FIELD_SIZE
32:   Create tmp file and log for blocks
33:   Reply with upload plan (key, size, block info)
34: end if
35: Server OP_UPLOAD (file_process):
36: if request_operation = OP_UPLOAD then
37:   Validate key, block_index, and data
38:   Write block into tmp file and record index
39:   if all blocks received then
40:     Merge tmp file and reply "All blocks uploaded"
41:   else
42:     Reply "This block uploaded"
43:   end if
44: end if
```
